
Línea de Énfasis en Ciencia de Datos y Analítica
Proyecto Integrador 1

ShiftMetrics Analytics

*Sistema Analítico Basado en Métricas de Ciclos de Desarrollo
de Software para la Adopción de Shift-Left Testing*

Equipo:	ShiftMetrics Analytics
Integrante 1:	Samuel Andrés Ariza Gómez
Integrante 2:	Juan Pablo Mejía Pérez
Integrante 3:	Fredy Cadavid Franco
Integrante 4:	Juan Pablo Rua Cartagena
Integrante 5:	Pedro Saldarriaga Fontán
Fecha de entrega:	3 de mayo de 2026

- Curso 1: SI7006 , Almacenamiento y Procesamiento de Grandes Datos
Curso 2: SI7009 , Aprendizaje Automático
Curso 3: SI7007 , Visualización de Datos
-

Descripción General de la Industria, Negocio y Problema

La industria global del desarrollo de software enfrenta una crisis estructural de calidad cuyo costo económico es medible y creciente. Según el *Consortium for Information and Software Quality* (CISQ, 2022), el software de baja calidad le costó a los Estados Unidos más de \$2.41 billones de dólares en 2022, siendo los defectos detectados en etapas tardías del ciclo de vida del desarrollo de software (SDLC) uno de los factores más determinantes de ese costo. Esta problemática es transversal a sectores como fintech, salud digital, comercio electrónico y servicios de tecnología, y es especialmente aguda en organizaciones que operan bajo esquemas de entrega continua (CI/CD), donde la velocidad de despliegue amplifica el impacto de cada defecto que escapa hacia producción.

El paradigma Shift-Left Testing (SLT) emerge como respuesta estratégica a este problema: propone mover las actividades de prueba y verificación de calidad hacia las fases más tempranas del SDLC, mucho antes de que los defectos se propaguen a entornos productivos. La evidencia empírica es consistente, un defecto detectado en producción cuesta entre 10 y 100 veces más corregirlo que uno detectado durante la codificación (Boehm, 1981; McConnell, 2004), y los equipos de alto rendimiento catalogados como *Elite* según el modelo DORA (Forsgren et al., 2018) aplican sistemáticamente prácticas de calidad tempranas como parte de su cultura de ingeniería.

Sin embargo, la adopción efectiva del SLT exige algo que la mayoría de organizaciones no tiene: visibilidad analítica, basada en datos, sobre su propio proceso de desarrollo. Las herramientas de gestión de proyectos ágiles, Jira Software, Azure DevOps, Linear, acumulan trazas longitudinales ricas del proceso: historias de usuario, bugs, sprints, tiempos de ciclo, prioridades y transiciones de estado. Estos registros constituyen una fuente de datos de alto valor que, en la gran mayoría de las organizaciones, permanece completamente subutilizada para la gestión de la calidad.

ShiftMetrics Analytics propone transformar esas trazas en inteligencia operativa: extraer y computar automáticamente métricas de proceso y calidad de estándares de facto (ISO/IEC 25010, métricas DORA, framework SPACE, TMMi), entrenar modelos de aprendizaje automático sobre ciclos de desarrollo simulados con parámetros calibrados en datos reales, y construir un sistema de visualización interactivo orientado a la toma de decisiones estratégicas sobre la adopción de SLT en equipos de alto rendimiento.

Problema a Resolver

Declaración del problema

Los equipos de desarrollo de software de alto rendimiento no disponen de un sistema basado en datos que les permita cuantificar, predecir y comunicar el riesgo de escape de defectos a lo largo del SDLC. Como consecuencia, las decisiones sobre cuándo y cómo implementar prácticas de *Shift-Left Testing* se toman de forma reactiva, intuitiva y sin respaldo cuantitativo, lo que perpetúa ciclos de deuda técnica, retrabajo y entregas de baja calidad.

El problema tiene tres dimensiones concretas:

1. **Ausencia de métricas de proceso sistematizadas.** Los equipos carecen de un catálogo de métricas rigurosas , Bug-to-Story Ratio, Defect Escape Rate, Phase Containment Effectiveness, métricas DORA, Test Shift-Left Index, entre otras , computadas automáticamente a partir de sus propias trazas de trabajo en herramientas tipo Jira. Sin estas métricas, no es posible diagnosticar objetivamente el estado de madurez de calidad de un sprint ni del equipo en su conjunto.
2. **Ausencia de capacidad predictiva.** No existen modelos de aprendizaje automático entrenados sobre datos de proceso de desarrollo que permitan anticipar, antes del cierre de un sprint, la probabilidad de que los ítems de trabajo en curso generen defectos escapados en fases posteriores del SDLC. Esta capacidad predictiva es la condición necesaria para que el SLT sea una estrategia proactiva y no una reacción tardía ante el fallo.
3. **Ausencia de comunicación ejecutiva basada en evidencia.** Los hallazgos de la analítica de calidad rara vez se traducen en narrativas accionables para los *stakeholders* (Product Owners, Engineering Managers, CTO), lo que impide que las recomendaciones de adopción de SLT escalen dentro de la organización y se conviertan en decisiones de inversión.

¿Por qué es importante este problema?

Impacto económico. La detección tardía de defectos es una de las formas más costosas e invisibles de desperdicio en los proyectos de software. En organizaciones con cadencias de despliegue diarias o semanales, cada defecto escapado a producción conlleva no solo el costo de corrección, sino también el de restauración del servicio, la pérdida de confianza del usuario y el daño reputacional. Cuantificar este costo mediante métricas objetivas es el primer paso para justificar la inversión en prácticas de calidad tempranas.

Impacto operacional. Los equipos de ingeniería toman decenas de decisiones por sprint sobre cómo distribuir el esfuerzo entre desarrollo, pruebas y corrección de deuda técnica. Sin datos que orienten estas decisiones, el resultado es sistemáticamente sub-óptimo. Un sistema que provea predicciones de riesgo accionables en el momento en que aún es posible intervenir , típicamente al 50% del sprint , tiene un impacto directo y medible sobre la eficiencia del equipo y la calidad del producto entregado.

Impacto estratégico. Las organizaciones líder en madurez de ingeniería de software , clasificadas como *Elite* o *High* en el modelo DORA , invierten sistemáticamente en instrumentación analítica de su proceso de desarrollo. La brecha entre estas organizaciones y las de menor madurez es, en gran medida, una brecha de datos y de capacidad analítica. ShiftMetrics Analytics propone un camino concreto, basado en estándares de la industria y en aprendizaje automático, para que los equipos cierren esa brecha.

Conjuntos de Datos Considerados

La estrategia de datos combina tres datasets abiertos de reconocida trayectoria académica , verificados y accesibles al momento de elaborar este documento , con un dataset sintético generado por el equipo mediante un motor de simulación calibrado sobre los datos reales.

Dataset 1 , PROMISE Defect Dataset (GitHub, público)

Colección de datasets de predicción de defectos de software ampliamente utilizada en la literatura de ingeniería de software empírica. Contiene métricas de código fuente a nivel de módulo y clase (métricas de Halstead, McCabe, Chidamber & Kemerer) junto con etiquetas binarias de defectuosidad, provenientes de proyectos reales open-source de Apache: Ant, Camel, Ivy, JEdit, Log4j, Lucene, POI, Synapse, Velocity, Xalan y Xerces, en múltiples versiones. Este dataset es el estándar de facto para benchmarking en predicción de defectos y es citado en cientos de publicaciones del área.

Acceso: <https://github.com/feiwww/PROMISE-backup> , público, descarga directa, sin registro.

Licencia: Académica abierta.

Dataset 2 , Apache Jira Issue Tracking Dataset (Zenodo, abierto)

Dataset publicado por Diamantopoulos y Symeonidis (Universidad Aristóteles de Tesalónica) que contiene el registro completo del sistema de seguimiento de issues de la Apache Software Foundation, descargado mediante la API oficial de Jira. Incluye colecciones de issues, comentarios, eventos de transición de estado, worklogs y proyectos en formato MongoDB dump. Cubre proyectos de gran escala como Hadoop, Spark, Kafka, Hive, Tomcat y decenas más, con campos relevantes para el cómputo de métricas de flujo: tipo de issue, prioridad, estado, resolución, fechas de creación y cierre, componentes y tiempo registrado.

Acceso: <https://zenodo.org/records/5665896> , abierto, descarga directa (2 GB comprimido).

Licencia: Creative Commons Attribution 4.0 International (CC BY 4.0).

Dataset 3 , Red Hat Public Jira Dataset 2001–2024 (Zenodo, abierto)

Dataset publicado en diciembre de 2024 por investigadores de las universidades de Limerick, Galway y Carnegie Mellon. Contiene datos de proceso de 490.000 Product Backlog Items (PBIs) distribuidos en 238 sistemas de desarrollo de software del portafolio público de Red Hat (issues.redhat.com), abarcando un período de 23 años (2001–2024). Los campos disponibles son: clave del issue, tipo, estado, proyecto, resolución, fecha de creación y fecha de cierre. Es especialmente valioso para el análisis de patrones de proceso a largo plazo y para el cómputo de métricas de flujo agregadas por sistema y período.

Acceso: <https://zenodo.org/records/14558684> , abierto, descarga directa (266 MB).

Licencia: Creative Commons Attribution 4.0 International (CC BY 4.0).

Dataset 4 , GHArchive: Eventos Públicos de GitHub

Archivo público de eventos de repositorios GitHub en tiempo real (commits, pull requests, issues, releases, deployments), disponible desde 2011 en archivos JSON por

hora. Permite calcular proxies de métricas DORA , en particular *Deployment Frequency* y *Change Failure Rate* , a partir de commits y PRs etiquetados como correcciones de errores (**fix**, **hotfix**, **bug**). Los datos de los proyectos del Dataset 1 y 2 (Apache) pueden cruzarse con sus repositorios GitHub para enriquecer el análisis.

Acceso: <https://www.gharchive.org/> , público, descarga directa por rango de fechas, también disponible en Google BigQuery Public Datasets.

Licencia: Creative Commons Attribution 4.0 International (CC BY 4.0).

Dataset 5 , Datos Simulados: Motor de Simulación de Ciclos de Desarrollo (generación propia)

Motor de simulación desarrollado por el equipo en Python (usando SimPy, Faker y NumPy) que genera sprints sintéticos tipo Jira con parámetros calibrados a partir de los datasets anteriores. Simulará un mínimo de 50 sprints bajo tres escenarios de madurez de calidad contrastados: (a) equipo sin SLT, con pruebas concentradas en etapas finales y alta tasa de escape de defectos; (b) equipo con SLT parcial, pruebas en CI/CD, tasa media; y (c) equipo con SLT maduro, pruebas desde diseño, tasa baja. Cada registro de sprint incluirá más de 40 métricas de proceso, producto y calidad derivadas de estándares de facto (ISO/IEC 25010, DORA, SPACE, TMMi). Este dataset será publicado en un repositorio GitHub público bajo licencia MIT al concluir el proyecto.

Apertura del Problema y los Datos

El problema abordado es de dominio abierto y de alta relevancia tanto en la comunidad académica de ingeniería de software empírica como en la industria de tecnología. No está circunscrito a ninguna empresa, sector o geografía específica: la gestión de la calidad en el SDLC mediante datos es un reto universal para cualquier equipo de desarrollo de software que opere bajo metodologías ágiles.

Los datasets 1, 2, 3 y 4 son completamente abiertos, de descarga directa y sin restricciones comerciales, tal como se verificó al momento de elaborar este documento. El dataset 5 es generado íntegramente por el equipo y se publicará en abierto. El código fuente completo del proyecto , pipeline de datos, motor de simulación, modelos y dashboard , se publicará bajo licencia MIT en un repositorio GitHub público al finalizar el trabajo.

Restricciones del Proyecto

- **En datos:** Los datasets 2 y 3 pueden contener en versiones anteriores información de identificación personal de desarrolladores (nombres de usuario, correos electrónicos). El equipo utilizará únicamente las versiones anonimizadas disponibles en Zenodo y operará exclusivamente con métricas agregadas a nivel de sprint y equipo, nunca a nivel individual. Queda explícitamente excluido cualquier análisis de productividad o desempeño individual de desarrolladores.

- **En código:** Todo el código del proyecto se desarrollará en Python y se publicará bajo licencia MIT. Las dependencias de terceros (Apache Spark, Scikit-learn, XGBoost, MLflow, Streamlit, FastAPI) son de código abierto con licencias compatibles. El entrenamiento de modelos se ejecutará en infraestructura gratuita o de acceso académico: Databricks Community Edition, Google Colab o equivalentes, para garantizar la reproducibilidad sin costo adicional.
- **En divulgación:** Los resultados del proyecto son divulgables en su totalidad. Los datos sintéticos generados por el simulador se compartirán públicamente como contribución a la comunidad de investigación en ingeniería de software empírica, acompañados de la documentación necesaria para su reproducción.
- **En uso ético:** Los modelos de ShiftMetrics Analytics están diseñados para apoyar decisiones sobre el *proceso* de desarrollo a nivel de equipo o sprint, no para evaluar el desempeño individual de ningún desarrollador. Este principio es no negociable y se documentará explícitamente en el repositorio del proyecto.
- **En propiedad intelectual:** Los datasets 1, 2, 3 y 4 cuentan con licencias abiertas que permiten su uso para investigación académica sin restricciones comerciales. No existe ningún conflicto de propiedad intelectual con empresas o terceros. Todo el material generado por el equipo se publicará en abierto.

Metodología del Proyecto y Plan de Trabajo

Para garantizar la rigurosidad y el éxito de *ShiftMetrics Analytics*, el desarrollo analítico se estructura siguiendo un enfoque basado en **CRISP-DM** (Cross-Industry Standard Process for Data Mining), el cual ha sido iterado y contextualizado a la ingeniería de software y operaciones MLOps. El ciclo de vida consta de las siguientes fases:

- **Definición del problema:** Caracterización formal del costo del retrabajo generado por defectos tardíos en el SDLC y el planteamiento de una solución analítica que cuantifique el riesgo para viabilizar la adopción de *Shift-Left Testing*.
- **Adquisición de datos:** Proceso de extracción e ingesta desde los repositorios de datos crudos (PROMISE, Apache/Red Hat Jira, GHArchive) hacia el entorno cloud, además de la ejecución del motor de simulación para generar el set de datos sintéticos.
- **Análisis exploratorio de datos (EDA):** Evaluación estadística inicial, análisis de distribuciones para tiempos de ciclo, identificación de outliers en las trazas de trabajo e imputación de variables proxy de la metodología DORA.
- **Preliminar de arquitectura de referencia:** Diseño esquemático del flujo y procesamiento bajo la Arquitectura de Medallón (Bronze, Silver, Gold), estableciendo las políticas de limpieza y de linaje de datos.

- **Tecnologías y potenciales servicios a emplear:** Provisión de la infraestructura sobre Google Cloud Platform, orquestando servicios distribuidos y de almacenamiento (Cloud Storage, Dataproc, BigQuery, Vertex AI).
- **Desarrollo de modelos supervisados y/o no supervisados:** Diseño de algoritmos de clasificación (basados en *Logistic Regression* y *XGBoost*) orientados a la predicción de la probabilidad de escape de defectos al 50% de la ejecución del ciclo.
- **Validación y Pruebas:** Evaluación algorítmica con segmentación temporal (*time-based split*) para evitar el *look-ahead bias*, priorizando la métrica F2-Score para penalizar los falsos negativos.
- **Visualización de datos y Analítica visual:** Creación de interfaces interactivas en Streamlit. Incluye interpretación técnica con *SHAP values* para los líderes de equipo y resúmenes de impacto (ROI) para la alta gerencia.
- **Pruebas (Integrales y E2E):** Evaluación de latencia desde el modelo hasta el dashboard (<60s) y certificación de integridad de las tuberías de datos en la nube.
- **Difusión y comunicación del proyecto:** Empaquetamiento final del sistema, liberación open-source bajo licencia MIT, publicación de datos y presentación formal de la viabilidad técnica.

Cronograma por semana del proyecto: mes de mayo y primera semana de junio

En la siguiente tabla se proyecta el cronograma general abarcando un total de 8 semanas, distribuyendo la carga de las actividades metodológicas detalladas anteriormente.

Actividades del Proyecto	W1	W2	W3	W4	W5
Definición del problema e ingesta de datos	X				
Análisis Exploratorio de Datos (EDA)	X				
Arquitectura Cloud y Pipeline (Bronze/Silver)	X	X			
Desarrollo Capa Gold y Modelado Supervisado		X	X		
Validación algorítmica y optimización			X	X	
Analítica visual y Dashboard interactivo			X	X	
Pruebas integrales, difusión y cierre				X	X

Arquitectura Tecnológica y Pipeline de Datos (GCP Stack)

El proyecto adopta una **arquitectura Medallion** (Bronze → Silver → Gold) implementada sobre la infraestructura de **Google Cloud Platform (GCP)**. Se utiliza **Google Cloud Storage (GCS)** como el Data Lake central, garantizando alta disponibilidad y durabilidad para los cinco datasets integrados. El motor de procesamiento principal es **Apache Spark** (vía Dataproc), seleccionado por su capacidad para gestionar transformaciones distribuidas y cargas de trabajo en tiempo real.

Capas de la arquitectura y Flujo de Datos

Capa Bronze — Ingesta y Raw Storage (GCS)

Los datos se ingieren en su formato nativo. Los Datasets 1 y 3 (CSV) se transfieren mediante **Cloud Storage Transfer Service**. El Dataset 2 (MongoDB dump) se normaliza a Parquet mediante jobs de **Dataproc**. El Dataset 4 (GHArchive) se consume mediante **Spark Structured Streaming**, aplicando ventanas temporales para la captura de eventos de GitHub en cuasi-tiempo real.

Capa Silver — Procesamiento y Curaduría

Se aplican procesos de limpieza, *schema enforcement* y normalización de tipos. En esta capa se realiza el **Feature Engineering** para computar métricas de proceso (Cycle Time, PCE y proxies DORA). La persistencia se realiza en tablas de BigQuery utilizando el **BigQuery Storage Write API** para garantizar la integridad y velocidad de escritura.

Capa Gold — Entrega y Analítica (BigQuery)

Capa de consumo final optimizada mediante **particionamiento por tiempo y clustering por proyecto**, minimizando la latencia y el costo de escaneo. Actúa como fuente de verdad para el entrenamiento de modelos en **Vertex AI** y el servicio de métricas al dashboard de Streamlit.

Diseño del Experimento de Aprendizaje Automático

Modelado Predictivo y Evaluación

Se define una tarea de **clasificación binaria supervisada** para predecir el riesgo de escape de defectos al punto medio del sprint.

- **Estrategia de Validación:** Se implementa un **split temporal estricto** (70/15/15) para evitar el sesgo de información futura (*look-ahead bias*). Adicionalmente, se aplica validación cruzada *Leave-One-Project-Out* para asegurar que el modelo sea agnóstico al proyecto evaluado.
- **Optimización de Métricas:** El modelo base (Regresión Logística) se comparará contra un ensamble de **XGBoost**. La métrica primaria de optimización

es el **F2-Score**, priorizando la reducción de falsos negativos para evitar que sprints de alto riesgo pasen desapercibidos, impactando directamente en la reducción del **Costo de Retrabajo**.

Diseño del Sistema de Visualización

El sistema de visualización se desarrolla en **Streamlit**, orquestando una narrativa de datos que responde a la necesidad de proactividad en la calidad del software.

- **Accionabilidad Técnica:** La vista para *Tech Leads* integra **SHAP values**, permitiendo que el usuario identifique exactamente qué métricas están elevando el riesgo en el sprint actual (ej. baja frecuencia de despliegue o alta complejidad ciclomática) y tome acciones correctivas inmediatas.
- **Impacto de Negocio:** La vista ejecutiva presenta una comparativa de escenarios de madurez SLT, traduciendo las probabilidades del modelo a un **ROI proyectado**, permitiendo a los *stakeholders* visualizar el ahorro económico de una adopción temprana de pruebas.

Criterio de validación: El sistema es satisfactorio si permite identificar factores de riesgo críticos en menos de 60 segundos mediante los paneles interactivos de Streamlit conectados a la infraestructura de GCP.

Referencias

- [1] Boehm, B. W. (1981). *Software Engineering Economics*. Prentice-Hall.
- [2] Consortium for Information and Software Quality (CISQ). (2022). *The Cost of Poor Software Quality in the US: A 2022 Report*. Synopsys.
- [3] Diamantopoulos, T., & Symeonidis, A. (2021). *Apache Jira Issue Tracking Dataset* [Data set]. Zenodo. <https://zenodo.org/records/5665896>
- [4] Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. IT Revolution Press.
- [5] GHArchive. (2011). *GH Archive: Eventos Públicos de GitHub*. Recuperado de <https://www.gharchive.org/>
- [6] McConnell, S. (2004). *Code Complete: A Practical Handbook of Software Construction* (2da ed.). Microsoft Press.
- [7] Menzies, T., et al. (2005). *The PROMISE Repository of Software Engineering Databases*. <https://github.com/feiwww/PROMISE-backup>
- [8] Investigadores de las universidades de Limerick, Galway y Carnegie Mellon. (2024). *Red Hat Public Jira Dataset 2001–2024* [Data set]. Zenodo. <https://zenodo.org/records/14558684>